

OS_PROJECT02_10831_2021026108

컴퓨터소프트웨어학부 2021026108 김민제

notion을 이용하여 작성한 리포트입니다.

Design

이번과제를 시작하기에 앞서 지난번 과제에서 제공된 xv6 파일과 달라진 점을 찾았습니다. 먼저 kill 함수 아래에 setkilled라는 함수가 추가되었는데 그냥 단순히 함수 내부에서 lock을 잡느냐 아니면 직접 잡느냐의 차이였습니다. 스레드를 구현하는데서 가장 중요한 변경점은 tramframe_va의 추가였습니다. 이것이 추가가 되며 pagetable을 만들고 proc을 만들거나 정리하는 부분에서 trapframe_va를 사용하여 동작하여 이에 대해서 이해하여 코드를 구현했습니다. 스레드의 경우 pagetable을 공유하여 사용하여 이를 연결해주기 위해서 trapframe_va 에 값을 저장하여 이를 이용하여 mappages()로 pagetable에 연결해주었습니다.

1. Thread 구현을 위해서는 clone과 join 시스템콜을 구현해야해서 이를 구현하기 위해서 fork와 wait를 확인하며 구현했습니다.
2. clone 구현을 위해서 PGSIZE 만큼 stack을 할당하여 ustack을 만들어 사용하기로 했습니다.
3. allocproc 대신 allocthread를 만들어서 trapframe을 새로 만들어주고 pagetable은 공유하도록 설계했습니다.
4. clone에서 allocthread를 호출하여 빈 thread를 만들고 clone에서 전달받은 stack을 ustack으로 설정하고 trapframe에 필요한 값을 할당하도록 했습니다.
5. join에선 이렇게 만들어진 thread를 찾아서 freethread 함수를 호출해서 할당한 공간과 mapping 한 table들을 정리해서 메모리를 관리했습니다.
6. 그 외의 추가적으로 kill, growproc, freeproc, exec 등을 수정하여 추가적인 시스템콜도 올바르게 동작하도록 했습니다.

Implementation

1. ULIB를 위한 user/thread.c 와 user/thread.h 구현

- a. thread.h에서는 필요한 헤더파일과 함수 선언을 해주었습니다.
- b. thread.c에서는 thread_create, thread_join 함수를 정의했습니다.
 - a. thread_create에서 stack에 PGSIZE*2만큼 할당을 해주고 clone 시스템콜을 호출합니다. 이때 오류가 발생하면 -1을 반환하고 정상적으로 동작한다면 tid(스레드의 id)를 반환하도록 했습니다.

```
if ((stack= malloc(2*PGSIZE)) == 0)
    return -1;
```

- b. thread_join의 경우 join 시스템콜을 호출하여 ustack을 반환받아 반환받은 stack을 정리해줍니다.
- c. makefile에 ULIB = \$U/thread.o를 추가하여 ulib에 추가해주었습니다.

2. clone, join 시스템콜 등록

- a. user/user.h에 clone, join 함수를 선언하여 사용할 수 있도록 합니다.

```
int clone(void (*fcn)(void*, void*), void *arg1, void *arg2, void *stack);
int join(void **stack);
```

- b. kernel/syscall.h 와 kernel/syscall.c에 아래 처럼 syscall 번호를 등록합니다.

```
#define SYS_clone 22
#define SYS_join 23
extern uint64 sys_clone(void);
extern uint64 sys_join(void);
```

```
[SYS_clone] sys_clone,
[SYS_join] sys_join,
```

c. kernel/sysproc에서 인자를 받아서 clone, join 함수를 실행하고 값을 반환하도록 합니다.

```
uint64
sys_clone(void)
{
    uint64 fcn, arg1, arg2, stack;

    argaddr(0, &fcn);
    argaddr(1, &arg1);
    argaddr(2, &arg2);
    argaddr(3, &stack);
    return clone((void (*)(void*, void*))fcn, (void*)arg1,
                (void*)arg2, (void*)stack);
}

uint64
sys_join(void)
{
    uint64 ustack;

    argaddr(0, &ustack);
    return join((void **)ustack);
}
```

3. proc.c 구현 ⇒ thread 생성 및 종료를 위한 구현

a. proc.h에 추가로 필요한 변수들을 선언했습니다.

```
int is_thread; // 스레드인지 확인용
struct proc *main; // 메인 스레드 관리용
void *ustack; // 유저 스택 관리용
int tid; // thread id
```

b. kernel/defs.h에 proc.c의 함수들을 선언해서 다른 kernel 파일들도 사용할 수 있도록 했습니다.

```
// project02
int clone(void (*fcn)(void*, void*), void *arg1,
          void *arg2, void *stack);
int join(void **stack);
struct proc* allocthread(struct proc *parent);
void freethread(struct proc *p);
```

c. allocthread 구현

- proc을 순회하면서 비어있는 proc을 찾아서 사용 상태로 바꾸고 메인스레드를 인자로 받아서 각 스레드에 main을 설정해 주고 pid, parent 등도 main을 따라가도록 했습니다.
- tid는 처음에는 따로 allocpid와 비슷하게 allostid를 생성하여 할당했지만 테스트 코드의 결과의 값과 동일하게 맞추다보니 그냥 tid에 allocpid로 값을 할당했습니다.
- trapframe을 개별적으로 가지고 있어야해서 새로 만들어주고 pagetable은 부모와 공유하므로 포인터만 연결해주었습니다.

- d. 이를 연결하고 trapframe_va에 tid를 기준으로 위치를 다르게해서 mapping 시켜주었습니다. 그후 돌아갈 주소와 스택 포인터를 설정해주었습니다.

```
found:
    np->state = USED;
    np->parent = main->parent;
    np->pid = main->pid;
    np->main = main;
    np->is_thread = 1;

    np->tid = allocpid(); // 명세와 동일하게 맞추려면 pid 를 그냥 줌

    if ((np->trapframe = (struct trapframe*)kalloc()) == 0) {
        freethread(np);
        release(&np->lock);
        return 0;
    }

    np->pagetable = main->pagetable;
    np->sz = main->sz;

    np->trapframe_va = TRAPFRAME - (PGSIZE * np->tid);
    if (mappages(np->pagetable, np->trapframe_va,
                PGSIZE, (uint64)np->trapframe, PTE_R|PTE_W) < 0) {
        freethread(np);
        release(&np->lock);
        return 0;
    }

    np->ustack = 0;

    memset(&np->context, 0, sizeof(np->context));
    np->context.ra = (uint64)forkret;
    np->context.sp = np->kstack + PGSIZE;

    return np;
}
```

- d. clone 함수에서 실질적인 값들을 세팅해주었습니다.

- a. 특히 stack을 정렬하여 사용하라는 명세가 있어서 PGROUNDDOWN이라는 함수(매크로)로 stack을 정렬했습니다.
- b. trapframe을 연결해주고 arg1, arg2를 trapframe에 넣어주고 실행할 함수도 epc에 넣어주고 스택 포인터를 정렬된 스택 주소에 사이즈만큼 추가해서 세팅해주었습니다.

```
np->ustack = (void*)((uint64)stack);
uint64 alignstack = (PGROUNDDOWN((uint64)stack));

*(np->trapframe) = *(p->trapframe);
np->trapframe->sp = alignstack + PGSIZE * 2;
np->trapframe->a0 = (uint64)arg1;
np->trapframe->a1 = (uint64)arg2;
np->trapframe->epc = (uint64)fcn;
... // 이하는 fork와 동일(거의 유사)
```

- e. join 함수는 wait 함수를 가져와서 필요한 부분들을 수정해서 만들었습니다.

- a. 계속해서 돌면서 동작하고 있는 메인스레드를 제외하고 스레드를 정리해주었습니다.
- b. 이미 종료된 상태라면 ustack을 정리해주고 freethread를 호출해서 해당 스레드를 정리해주도록했습니다.

```
for(t = proc; t < &proc[NPROC]; t++){
    if(t!=p && t->main == p){
        acquire(&t->lock);
        havethread = 1;
        if(t->state == ZOMBIE){
            tid = t->tid;
            if (stack != 0 && copyout(p->pagetable, (uint64)stack,
                (char *)&t->ustack, sizeof(void *)) < 0) {
                release(&t->lock);
                release(&wait_lock);
                return -1;
            }
            freethread(t);
            release(&t->lock);
            release(&wait_lock);

            return tid;
        }
        release(&t->lock);
    }
}
```

- f. 마지막으로 freethread는 freeproc을 참고하여 만들었습니다.
 - a. 스레드가 아닌데 호출되었다면 에러메시지를 출력하고 pagetable을 unmap하고 trapframe을 free시켜 정리해줍니다.
 - b. 그 후 기본 값들을 0으로 초기화 시켜줍니다.

```
if (t->pagetable && t->trapframe_va) {
    uvmunmap(t->pagetable, t->trapframe_va, 1, 0);
    t->trapframe_va = 0;
}

if(t->trapframe){
    kfree((void*)t->trapframe);
    t->trapframe = 0;
}
...
t->is_thread = 0;
t->main = 0;
t->ustack = 0;
t->tid = 0;
```

4. 시스템콜 동작을 위한 수정들

- a. proc.c의 allocproc함수
 - a. main을 자기자신으로 설정하고, 스레드가 아니며 tid를 0으로 설정하도록 했습니다.
 - b. main thread는 allocproc을 통해서 생성이되기 때문에 main을 자기 자신으로 설정하는 부분이 반드시 필요했습니다.

```
p->main = p;
p->tid = 0;
p->is_thread = 0;
```

b. proc.c의 freeproc

- 이 함수의 경우 kill 함수에서 호출되는 경우가 있는데 스레드의 경우 freethread를 통해 정리해주어야해서 스레드인 경우 freethread로 들어가도록 추가했습니다.
- 자기 자신은 freeproc에서 마지막에 정리되어야 하므로 자기 자신을 제외한 다른 스레드들을 잡아서 freethread를 호출하도록 했습니다.
- 이 부분을 따로 처리하지 않으면 자식 스레드들을 unmap 하지않은 상태로 메인을 unmap 하게 되어 leaf panic이 발생합니다.

```
for(struct proc* np = proc; np < &proc[NPROC]; np++){
    if(np != p && np->main == p){
        freethread(np);
    }
}
```

c. proc.c의 proc_freepagetable 과 exec.c의 exec

- exec 함수를 실행할 때 스레드를 모두 정리하고 그 함수를 호출한 스레드가 새로운 프로세스가 되어야하는데 이를 정리하는 과정에서 스레드들을 먼저 정리하지 않고 unmap 후 free를 하면 에러가 발생합니다.
- 그래서 이를 보장해주기 위해서 속해있는 모든 스레드들을 죽이고 정리를 해줍니다.
- 그후 동일하게 oldpagetable을 정리해줍니다.
- 이를 위해서 exec 함수에서 초기의 p와 중간의 ELF read 이후의 값들이 변경되어 trapframe_va 값을 잃어버리게 됩니다. 그래서 이 값을 전에 따로 저장해두어 정리를 해줍니다.

```
pagetable_t pagetable = 0, oldpagetable;
struct proc *p = myproc();
uint64 tva = p->trapframe_va;
```

- 이후 user image를 commit하고 스레드라면 oldpagetable을 정리하기 위해서 저장해둔 va 값을 이용해서 정리해주고 proc_freepagetable을 호출해줍니다.

```
if (p->is_thread) {
    if (tva && walk(oldpagetable, tva, 0) != 0) {
        uvmunmap(oldpagetable, tva, 1, 0);
    }
    p->is_thread = 0;
    p->main = p;
    proc_freepagetable(oldpagetable, oldsz);
}else{
    proc_freepagetable(oldpagetable, oldsz);
}
```

- proc_freepagetable을 실행할 때 스레드들을 정리하기 위해서 스레드인 경우와 아닌 경우를 나누어처리를 해주었습니다. 단순히 스레드가 종료되어 정리를 하며 이를 실행할 때는 다른 스레드들을 모두 죽이며 정리해버리면 안되기 때문입니다.

```
if(!p->is_thread){
    for(struct proc* t = proc; t < &proc[NPROC]; t++) {
        if(t->is_thread && t->pid == p->pid){
            setkilled(t);
            if(t->main != p)
                freethread(t);
        }
    }
}
```

```

    uvmunmap(pagetable, TRAMPOLINE, 1, 0);
    uvmunmap(pagetable, TRAPFRAME, 1, 0);
    uvmfree(pagetable, sz);
}

```

d. proc.c의 growproc

- a. sbrk의 경우 proc.c의 growproc 함수를 호출하는데 이때 원래는 $sz = p \rightarrow sz$ 인데 스레드는 main을 따라가야하므로 $main \rightarrow sz$ 로 변경해주어 값을 올바르게 받도록 했습니다.

```

int
growproc(int n)
{
    uint64 sz;
    struct proc *p = myproc();
    struct proc *main = p->main;
    sz = main->sz;
    if(n > 0){
        if((sz = uvmalloc(p->pagetable, sz, sz + n, PTE_W)) == 0) {
            return -1;
        }
    } else if(n < 0){
        sz = uvmdealloc(p->pagetable, sz, sz + n);
    }
    main->sz = sz;
    return 0;
}

```

e. proc.c의 exit

- a. 스레드나 프로세스를 종료할 때 남아있는 스레드가 있으면 안되서 종료하는 프로세스(스레드)의 main과 동일하면 모두 setkilled로 죽이도록 했습니다.
- b. 그리고 wakeup을 호출할 때 스레드라면 $p \rightarrow main$ 을 깨우고 아니라면 부모를 깨워야해서 이 부분도 추가했습니다.

```

void
exit(int status)
{
    struct proc *p = myproc();
    for(struct proc *np = proc; np < &proc[NPROC]; np++) {
        if(np->main == p){
            setkilled(np);
        }
    }
    ...
    if(p->is_thread)
        wakeup(p->main);
    else
        wakeup(p->parent);
}

```

f. proc.c의 kill

- a. kill 동작의 경우 thread라면 모든 thread를 찾아서 죽여야하므로 입력받은 pid 기준으로 모든 proc을 순회하여 kill flag를 설정해줍니다.
- b. 이 함수가 종료가 되면 kill flag를 보고 kill을 합니다.
- c. 그래서 killing을 체크해서 죽인게 있다면 0, 없다면 -1을 반환하도록 했습니다.

```

int
kill(int pid)
{
    struct proc *p;
    int killing = 0;
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        if(p->pid == pid){
            p->killed = 1;
            killing++;
            if(p->state == SLEEPING){
                // Wake process from sleep().
                p->state = RUNNABLE;
            }
        }
        release(&p->lock);
    }
    if(killing > 0) return 0;
    else return -1;
}

```

g. proc.c의 alloctid

- a. tid에 값을 할당하기 위해서 allocpid와 동일하게 spinlock도 만들어 값을 할당하도록 했습니다.
- b. 하지만 명세와 결과가 달라 만들어놓고 사용은 하지 않았습니다.
- c. tid가 다르기만 하면 되서 크게 상관있는 문제는 아니었습니다.

```

int nexttid = 1;
struct spinlock tid_lock;

```

Results

makefile에 테스트 파일들을 등록해주고 테스트 파일들을 실행시켜 테스트를 진행했습니다.

```

$U/_thread_test\
$U/_thread_fcn

```

제공받은 테스트 파일을 통해서 모든 명령들이 올바르게 동작하는지 확인했습니다.

```

$ thread_test

[TEST#1]
Thread 0 start
Thread 1 start
Thread 1 end
Thread 2 start
Thread 2 end
Thread 3 start
Thread 3 end
Thread 4 start
Thread 4 end
Thread 0 end
TEST#1 Passed

[TEST#2]
Thread 0 start, iter=0
Thread 0 end
Thread 1 start, iter=1000
Thread 1 end
Thread 2 start, iter=2000
Thread 2 end
Thread 3 start, iter=3000
Thread 3 end
Thread 4 start, iter=4000
Thread 4 end
TEST#2 Passed

[TEST#3]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Child of thread 0 start
Child of thread 1 start
Child of thread 2 start
Child of thread 3 start
Child of thread 4 start
Child of thread 0 end
Child of thread 1 end
Child of thread 2 end
Child of thread 3 end
Child of thread 4 end
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#3 Passed

[TEST#4]
Thread 0 sbrk: old break = 0x0000000000015000
Thread 0 sbrk: increased break by 14000
new break = 0x000000000029010
Thread 1 size = 0x0000000000029010
Thread 2 size = 0x0000000000029010
Thread 3 size = 0x0000000000029010
Thread 4 size = 0x0000000000029010
Thread 0 sbrk: free memory
Thread 0 end
Thread 1 end
Thread 2 end
Thread 3 end
Thread 4 end
TEST#4 Passed

[TEST#5]
Thread 0 start, pid 29
Thread 1 start, pid 29
Thread 2 start, pid 29
Thread 3 start, pid 29
Thread 4 start, pid 29
TEST#5 Passed

[TEST#6]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Executing...
Thread exec test 0
TEST#6 Passed

All tests passed. Great job!!
$

```

그 결과 올바르게 동작하고 올바르게 종료가 되어 반복적으로 테스트 파일을 동작했을때도 이상없이 동작하는 것을 확인할 수 있었고 이후 usertests를 통하여 운영체제가 정상적으로 동작하는지도 확인했습니다.

```

[TEST#5]
Thread 0 start, pid 298
Thread 1 start, pid 298
Thread 2 start, pid 298
Thread 3 start, pid 298
Thread 4 start, pid 298
TEST#5 Passed

[TEST#6]
Thread 0 start
Thread 1 start
Thread 2 start
Thread 3 start
Thread 4 start
Executing...
Thread exec test 0
TEST#6 Passed

All tests passed. Great job!!
$

test sbrklast: OK
test sbrk8000: OK
test badarg: OK
usertests slow tests starting
test bigdir: OK
test manywrites: OK
test badwrite: OK
test execout: OK
test diskfull: balloc: out of blocks
balloc: out of blocks
OK
test outofinodes: ialloc: no inodes
OK
ALL TESTS PASSED
$

```

Trouble Shooting

1. 처음에 구조를 잡을 때 main을 사용하지 않고 그냥 parent를 이용해서 구현했는데 이렇게 하다보니 메인 스레드를 추적하는 것이 어려워서 main을 추가하여 사용했습니다. parent로 하다보니 예상치 못한 곳에서 에러가 많이 발생하였고 이로 인해서 다른 부분을 잘 짜놓고 돌아가지 않는 슬픈상황이 있었지만 잘 해결했습니다.
2. Kill과 관련된 문제
 - a. kill을 구현할 때 하나의 스레드만 죽이더라도 모든 스레드를 죽여야해서 예시 결과의 마지막 Thread 0 end가 절대 출력이 될 수 없다고 생각했습니다. 그러나 kill 명령의 동작이 즉시 반영되는 것이 아니라 kill flag만 설정하고 timer interrupt에 의해 정해진 timer 마다 동작하는 것이어서 코드의 상태나 컴퓨터의 상태에 따라서 출력이 될 수도 안될 수도 있다는 것을 알게 되었습니다.
3. 처음에는 sbrk 문제를 해결하기 위해서 leaf panic 문제에 크게 막혔었습니다. leaf panic은 unmap과 관련하여 freepagetable을 호출할 때 일어났습니다. mapping 되어있는 하위 pagetable을 제거하지 않고 unmap을 한다면 발생하는 에러였고 이를 해결하기 위해서 freeproc, freethread 등 free하는 부분에 대해서 찾아서 해결하려 했습니다. 하지만 해결되지 않았고 혹시 sz와 관련이 있지 않을까?라는 생각으로 sbrk 동작을 따라가다 growproc이라는 함수에서 sz를 증가시킬때 main이 아닌 자기 자신만을 업데이트하여 문제가 생긴다는 것을 알아차렸습니다.
4. 처음에는 gettid라는 함수를 새로 만들어서 pid와 tid를 별개로 두고 구현을 했는데 예시 결과에서는 그냥 pid를 tid처럼 사용하는 것이라고 생각하여 그냥 getpid로 새로운 tid를 만들도록 했습니다.

5. `exec` 문제를 해결할 때 동일한 프로세스의 스레드를 `freethread`를 통해서 정리를 하는데 자기 자신의 경우 `exec` 함수에 처음 들어왔을 때와中间的의 입출력 관리부분을 지나 두번째로 `myproc`을 호출하였을 때의 `trapframe_va`의 값이 달라져서 `unmap`과 관련한 문제가 발생했습니다. 이를 `exec`함수가 실행될 때의 `va` 값을 저장해두었다가 따로 `unmap` 해주는 방식으로 해결했습니다.